

Del ADN a la Proteína: Modelado Computacional del Dogma Central, Splicing Alternativo y Estabilidad Estructural

Informe técnico – Bioinformática

Fabio Nesta Arteaga Cabrera Pablo Cabeza Lantigua

Resumen

El dogma central de la biología molecular describe el flujo de información desde los ácidos nucleicos hasta las proteínas. En este trabajo se modelan de forma analítica y computacional sus tres etapas, replicación semi-conservativa, transcripción y traducción, sobre el gen de la insulina humana (*INS*, NM_000207.3), evaluando la importancia de la direccionalidad 5' → 3'. Se estudian además los mecanismos de diversificación proteica mediante el splicing alternativo del gen *FGFR2* y las restricciones de plegamiento de la mioglobina (PDB 1MBO). Finalmente se presenta un pipeline en Biopython que automatiza e informa de las tres etapas del dogma central. Este informe es la memoria del cuaderno Jupyter *adn-to-protein.ipynb*, cuyos seis ejercicios se corresponden con las secciones del documento (Cuadro 1).

1. Introducción

Los sistemas biológicos pueden entenderse como canales de transmisión de información con redundancia y corrección de errores, que traducen entre un alfabeto de cuatro símbolos (ADN={A,T,C,G}, ARN={A,U,C,G}) y un alfabeto de veinte aminoácidos. El objetivo de esta práctica es implementar y verificar las tres transformaciones del dogma central, replicación, transcripción y traducción, y analizar dos fenómenos que amplían la complejidad, el splicing alternativo y la sensibilidad estructural de las proteínas a mutaciones puntuales.

El modelo semiconservativo de la replicación, propuesto por Watson y Crick a partir de la estructura en doble hélice, fue confirmado por Meselson y Stahl (1958) mediante centrifugación en gradiente de densidad con isótopos de nitrógeno, demostrando que cada molécula hija conserva exactamente una hebra parental.

2. Metodología y herramientas

Se utilizó Python 3.14 con la librería Biopython sobre tres fuentes públicas de referencia, el registro NM_000207.3 de NCBI (gen *INS*, insulina humana), la base de datos Ensembl para las isoformas de *FGFR2* (ENSG00000066468) y el Protein Data Bank para la estructura de la mioglobina (1MBO). Todo el código se ejecutó en un cuaderno Jupyter (*adn-to-protein.ipynb*) versionado en el repositorio del proyecto.

El cuaderno sigue la misma numeración que el enunciado de la práctica, cada ejercicio contiene el desarrollo manual y, cuando se pide, su verificación con Biopython. El Cuadro 1 indica en qué ejercicio del cuaderno se encuentra cada resultado de este informe, y el código fuente completo está disponible en <https://github.com/ArtHead-Devs/ADN-to-Protein>.

Informe	Cuaderno	Contenido en el cuaderno
§ 3.1	Ej. 1	Hebras nuevas, enzimas; celda con <code>complement()</code>
§ 3.2	Ej. 2	Cadena molde y transcrito; celda que lee el FASTA y usa <code>reverse_complement()</code>
§ 3.3	Ej. 3	Codones y mutaciones; celda con <code>translate()</code>
§ 4.1	Ej. 4	Isoformas 1-2-3-5 y 1-3-5; análisis de <i>FGFR2</i> (Ensembl)
§ 4.2	Ej. 5	Extremos N y C; análisis de 1MBO (PDB)
§ 5	Ej. 6, punto 5	Reflexión sobre el punto más vulnerable
§ 6	Ej. 6, punto 6 + ext.	Función <code>pipeline_dogma_central()</code> y versión con mensajes de progreso

Cuadro 1: Correspondencia entre las secciones del informe y el cuaderno *adn-to-protein.ipynb*.

3. Replicación, transcripción y traducción: caso INS

3.1. Replicación semiconservativa

Sobre la secuencia 5'-ATGCCGTTAGCT-3' / 3'-TACGGCAATCGA-5', la replicación genera dos moléculas hijas idénticas, cada una con una hebra parental y una recién sintetizada. El proceso depende de cuatro enzimas, la helicasa rompe los puentes de hidrógeno entre hebras; la primasa sintetiza los cebadores de ARN que permiten iniciar la síntesis; la ADN polimerasa añade nucleótidos complementarios y corrige errores; y la ligasa sella los fragmentos de Okazaki de la hebra rezagada. Un error de la polimerasa no corregido se fija como mutación permanente, heredable a la descendencia celular.

Enzima	Función principal
Helicasa	Desenrolla y separa las hebras parentales
Primasa	Sintetiza cebadores de ARN (3'-OH libre)
ADN polimerasa	Elonga y corrige la hebra en síntesis
Ligasa	Sella los fragmentos de Okazaki

Cuadro 2: Enzimas de la horquilla de replicación.

La hebra complementaria se verificó con Biopython, comparando el resultado del método `complement()` frente al obtenido manualmente (celda de extensión del Ejercicio 1):

```
hebra_original = Seq("ATGCCGTTAGCT")
hebra_complementaria = hebra_original.complement()
resultado_manual = Seq("TACGGCAATCGA")
print(hebra_complementaria == resultado_manual)
# Salida: True
```

El resultado confirma que la complementariedad de bases ($A \leftrightarrow T$, $C \leftrightarrow G$) obtenida manualmente coincide exactamente con la calculada por la librería.

3.2. Transcripción direccional

La ARN polimerasa sintetiza siempre en sentido 5' → 3', por lo que lee la hebra molde en sentido 3' → 5' y sustituye T por U. Para 5'-ATGCCTGAATGC-3' / 3'-TACGGACTTACG-5', la cadena molde es 3'-TACGGACTTACG-5' y el transcrito resultante es 5'-AUGCCUGAAUGC-3'. La región promotora precede al gen y no se transcribe; la región codificante comienza en el codón AUG y contiene la información traducible.

Al invertir la orientación de la hebra (`reverse_complement().transcribe()`) se destruyen las señales biológicas, promotor y codón de inicio, y una traducción produciría un péptido no funcional. Sobre el ARNm de *INS* (NM_000207.3), la comparación entre ambas orientaciones ilustra esa pérdida de señal (salida Ejercicio 2, que lee `sequence.fasta` con `SeqIO`):

```
ARNm directo:  AGCCCUCCAGGACAGGCUGCAUC...
ARNm invertido: GCUGGUUCAAGGGCUUUAUUGCA...
```

El transcrito directo conserva el codón de inicio AUG en su marco de lectura original, mientras que el invertido presenta una composición de codones completamente distinta.

3.3. Traducción y ORF

Para el transcrito 5'-AUGUAUGCUUAA-3', el codón de inicio es AUG y el de paro UAA; la traducción produce Met-Tyr-Ala. Si AUG mutase a GUG, el ribosoma no reconocería el inicio; si desapareciera el codón de paro, el ribosoma continuaría hasta el siguiente codón de terminación, generando una proteína anormalmente larga.

La verificación automática con `Bio.Seq.translate()` reproduce exactamente el resultado obtenido manualmente, incluyendo el símbolo de parada cuando no se solicita su omisión (Ejercicio 3):

```
secuencia_arn = Seq("AUGUAUGCUUAA")
print(secuencia_arn.translate())           # MYA*
print(secuencia_arn.translate(to_stop=True)) # MYA
```

Aplicando el mismo procedimiento al ARNm de *INS* (NM_000207.3), el marco de lectura comienza en el primer AUG y termina en el codón de paro, obteniéndose el precursor de la insulina (Ejercicio 6, paso 3):

```
MALWMRLPLALLALWGPDPAAAFVNQHLGSHLVEALYLVCGERGFFYTPKTR
REAEDLQVGQVELGGGPGAGSLQPLALEGSLQKRGIVEQCCTSIICSLYQLENYCN
```

4. Diversidad transcripcional y estructural

4.1. Splicing alternativo: gen FGFR2

El splicing alternativo rompe la correspondencia “un gen = una proteína”: mediante el procesamiento combinatorio de exones, genera varias isoformas maduras de ARNm, siempre respetando el orden 5' → 3' (p.ej. 1-2-3-5 o 1-3-5, sin permutar exones). Estas isoformas pueden diferir en longitud, en la presencia de dominios funcionales, etc. El diseño de isoformas y la consulta a Ensembl se documentan en el Ejercicio 4 del cuaderno.

```
Pre-ARNm:  Exon1-Exon2-Exon3-Exon4-Exon5
Isoforma A: Exon1-Exon2-Exon3-----Exon5
Isoforma B: Exon1-----Exon3-----Exon5
```

Transcrito	ARN (nt)	Prot. (aa)	Observación
...358487.10	4.624	821	Isoforma de referencia
...457416.7	4.627	822	Un aa más; dominio distinto
...1142304.1	4.950	821	Igual proteína; UTR más larga
...369060.9	3.716	705	Pierde exones; función reducida
...683678.1	1.191	—	Intrón retenido; sin proteína

Cuadro 3: Isoformas del gen *FGFR2* (Ensembl, ENSG00000066468).

Del Cuadro 3 se extraen tres conclusiones: (1) proteínas de longitud casi idéntica pueden tener dominios distintos y por tanto afinidades diferentes; (2) transcritos de longitud muy distinta pueden codificar exactamente la misma proteína cuando la diferencia recae en regiones no traducidas, lo que afecta a la estabilidad del ARNm más que a la función proteica; y (3) la retención de intrones puede eliminar por completo la capacidad codificante, funcionando como mecanismo regulador que apaga la producción de una proteína sin desactivar el gen. En *FGFR2* concretamente, el intercambio alternativo de exones modifica la región de unión al ligando, permitiendo que el receptor responda a señales distintas según el tejido en el que actúe.

4.2. Estructura proteica: polaridad y mioglobina

Toda cadena tiene un extremo N-terminal (amino libre) y un extremo C-terminal (carboxilo libre); en Met-Ile-Ser-Gly-Val-Lys-His, el N-terminal es la metionina y el C-terminal la histidina. El orden de los aminoácidos determina las interacciones que estabilizan el plegamiento.

La mioglobina (PDB 1MBO, 153 aa) está formada casi por 8 hélices alfa que rodean el grupo hemo. Dos tipos de mutación son especialmente desestabilizantes. La inserción de una prolina en una hélice, cuya geometría rígida rompe el patrón regular de puentes de hidrógeno; y la sustitución de un aminoácido hidrofóbico del núcleo interno por uno hidrofílico, que introduce una carga en un entorno sin agua y desestabiliza el plegamiento, reduciendo la capacidad de unir oxígeno. Este análisis corresponde al Ejercicio 5 del cuaderno.

5. Vulnerabilidad a errores

Existe una asimetría entre frecuencia de error e impacto, la transcripción y la traducción fallan con mayor frecuencia, pero sus productos (ARNm, proteínas) son transitorios y se degradan sin dejar huella. La replicación del ADN, en cambio, es mucho más precisa gracias a la corrección de pruebas de la polimerasa, pero cualquier error no corregido se fija como mutación permanente, se transmite a la descendencia celular y contamina todos los transcritos y proteínas futuros de ese gen. Por ello, la replicación es el punto más crítico del proceso pese a ser el menos propenso a errores.

6. Pipeline computacional en Biopython

Se implementó una función que encadena las tres etapas e informa de cada paso (`pipeline_dogma_central()`, Ejercicio 6 del cuaderno):

```
def pipeline_dogma_central(fasta_file: str):
    registro = SeqIO.read(fasta_file, "fasta")

    # La secuencia del FASTA corresponde al ARNm,
    # por lo que
    # se convierte T/U para trabajar con una
    # secuencia de ADN.
    adn = Seq(str(registro.seq).replace("U", "T"))

    # 1. Replicacion
    hebra_cmplment = adn.complement()

    # 2. Transcripcion
    arnm = adn.transcribe()

    # 3. Traducción
    inicio = arnm.find("AUG")

    if inicio != -1:
        orf = arnm[inicio:]
        resto = len(orf) % 3

        if resto != 0:
            orf = orf[:-resto]

        proteina = orf.translate(to_stop=True)
    else:
        proteina = Seq("")

    print(f"PIPELINE DOGMA CENTRAL | ID:
          {registro.id} ({len(adn)} nt)\n")
    print(f"1. Replicacion (hebra
          complementaria):\n{hebra_cmplment[:60]}...\n")
    print(f"2. Transcripcion (ARNm
          5'->3'):\n{arnm[:60]}...\n")
    print(f"3. Traducción ({len(proteina)}
          aa):\n{proteina}")

pipeline_dogma_central("sequence.fasta")
```

La ejecución sobre `sequence.fasta` (registro NM_000207.3) produce la siguiente traza, confirmando que las tres etapas se completan de forma encadenada y trazable:

```
PIPELINE DOGMA CENTRAL | ID: NM_000207.3 (465 nt)

1. Replicacion (hebra complementaria):
TCGGGAGGTCTGTCCGACGTAGTCTTCTCCGGTAGTTCGTCTAGTGACAGGAA
GACGGT...

2. Transcripcion (ARNm 5'->3'):
AGCCCUCCAGGACAGGCUGCAUCAAGAGGCCAAGCAGAUACUGUCCUU
CUGCCA...

3. Traducción (110 aa):
MALWMRLPLLLALLALWGPDPAFAFVNQHLGSHLVEALYLVCGERGFFYTPKTR
REAEDLQVGQVELGGGPGAGSLQPLALEGSLQKRGIVEQCCTSICSLYQLENYCN
```

7. Conclusiones

El flujo de información genómica exige una coherencia direccional estricta ($5' \rightarrow 3'$); invertirla destruye las señales biológicas y anula su función. Además, el splicing alternativo permite diversificar el proteoma sin aumentar el número de genes, modulando afinidad, estabilidad de ARNm o regulación negativa.

Por otra parte, el plegamiento proteico depende de la correcta distribución de residuos hidrofóbicos e hidrofílicos y de la regularidad geométrica de sus hélices. Por último, la replicación del ADN es el punto más crítico del dogma central, sus errores, aunque poco frecuentes, son permanentes y heredables, a diferencia de los fallos transitorios de transcripción y traducción.

Repositorio y entregables

Repositorio: <https://github.com/ArtHead-Devs/ADN-to-Protein>

- `presentacion/`: carpeta con la presentación en PDF.
- `README.md`: esta misma memoria en formato Mark-down.
- `adn-to-protein.ipynb`: cuaderno Jupyter con el desarrollo manual, el código y las salidas de los seis ejercicios (ver Tabla 1).
- `sequence.fasta`: registro NM_000207.3 (gen *INS*), utilizado como entrada en las celdas de lectura FASTA y en el pipeline.
- `informe.pdf`: este documento, compilado desde LaTeX.
- `pyproject.toml`, `uv.lock` y `.python-version`: definición del entorno (Python 3.14 y dependencias, entre ellas Biopython) para reproducir los resultados con uv.
- `LICENSE`: licencia del proyecto.

Para reproducir los resultados basta con clonar el repositorio, crear el entorno con `uv sync` y ejecutar el cuaderno. La entrega consta de este informe (PDF y `README.md`), el código en formato `.zip` y el enlace al repositorio.